

VoiceChess

A voice-controlled robot to play chess

Embedded Systems, Cyber-Physical Systems and
Robotics (INHN0018)

Technical University of Munich

Campus Heilbronn

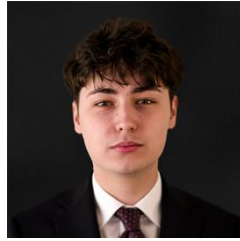
September 2026



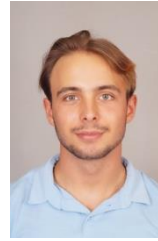
Team



Matei-Florian Tudose
Team Co-Lead
Electronics



David-Luca Mosu
Team Co-Lead
Mechanics and Design



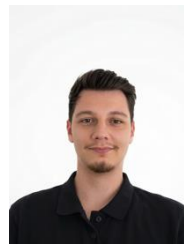
Anton Stoyanov
Software



Eni Ferizi
Software



David Enachescu-
Goldenberg
Software



David Gabriel-Marin
Software



Dorde Vidakovic
Software

Chess needs hands

A blind player cannot read the board state.

A player without hand mobility cannot move pieces.

Electronic boards still need a screen, an app, or a touch input.

Every option puts something between the player and the game.

Our target: play a complete game of chess using nothing but your voice.

Mechanics

Everything was made into CAD, then built

- Full assembly with joints and travel limits, we could move the robot on screen before printing anything
- Motion studies caught problems early: the first gripper swept into neighbouring squares and was replaced before it wasted filament
- Two sides of the machine are the same module, mirrored
- **~65% of the robot is 3D printed** (PETG), every bracket, mount, foot, the rack and the jaws
- **~35% is bought**, aluminum profile, motors, wheels, belts, screws, and one metal bar for the towers
- Rule we followed: buy what must be precise, print what must have a shape

How the robot moves

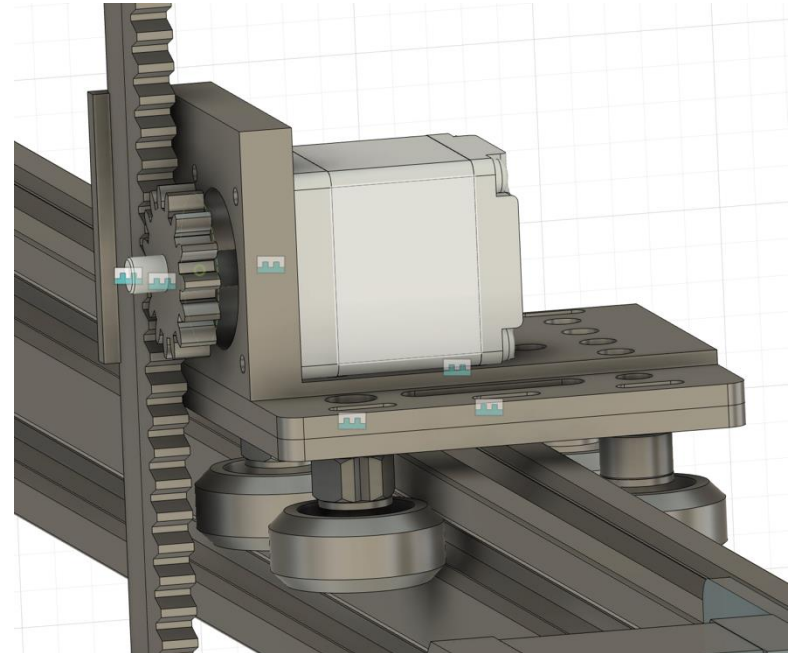
A 3-axis gantry, like a 3D printer

- **X** - two belt-driven carriages, one per rail, move the whole bridge front–back
- **Y** - a belt-driven carriage moves along the bridge left–right
- **Z** - a printed rack (an arm with gear teeth) is pushed down and pulled up by a pinion on the carriage
- A gripper at the bottom of the rack grabs the piece

The rails and wheels

Aluminium V-slot profile

- The beam is the frame and the rail at the same time
- Two 800 mm rails for Y, one 800 mm beam for X
- Carriages roll on plastic V-wheels

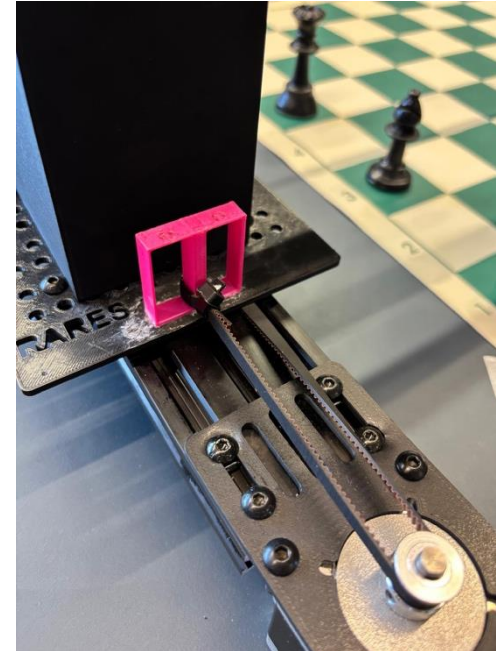
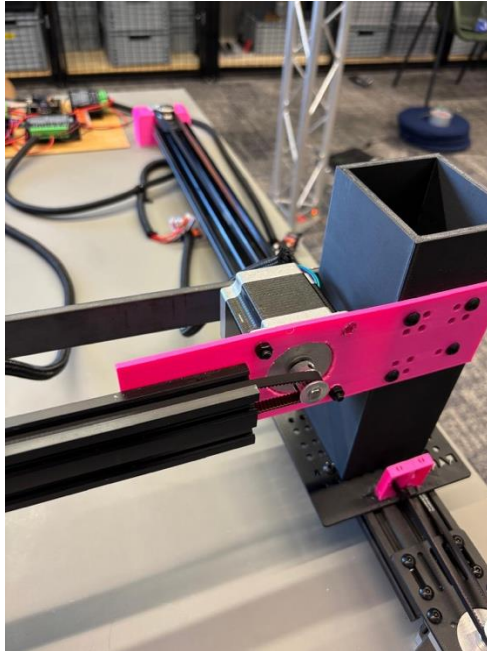


Motors and belts

Three NEMA 23 steppers, one NEMA 17

- Two NEMA 23 on Y, one on each rail, running in sync
- One NEMA 23 on X, mounted on the tower
- GT2 belts move X and Y; the belts are clamped and tensioned with zip ties
- NEMA 17 with a 5:1 gearbox drives Z

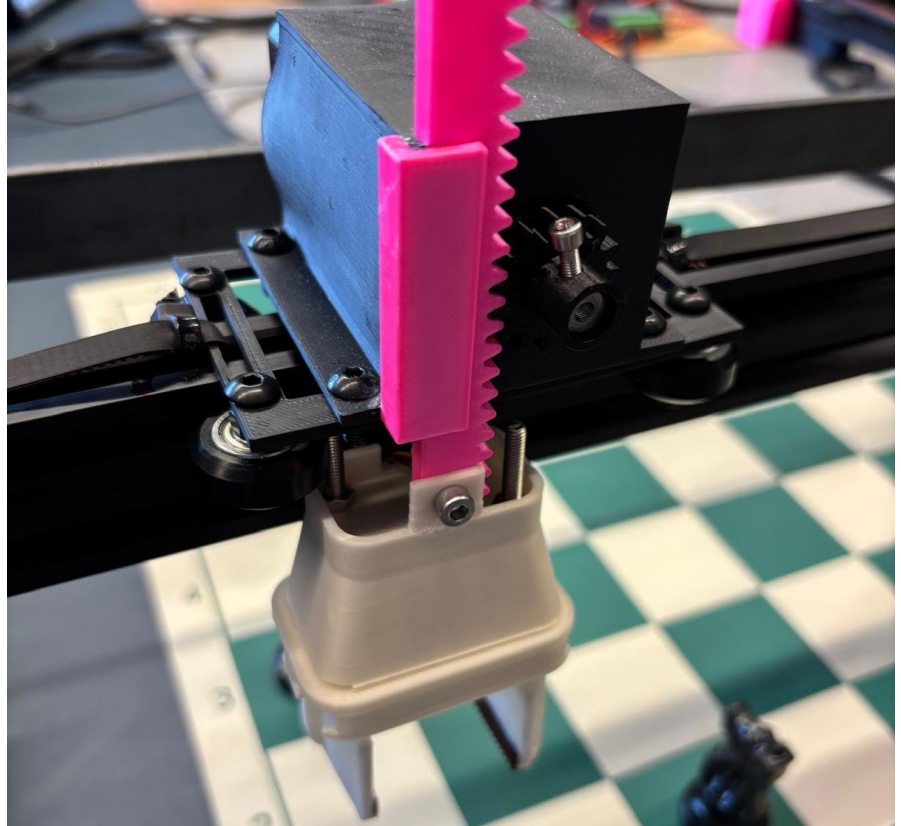
Motors and belts



The Z axis: the arm is a gear rack

Rack and pinion

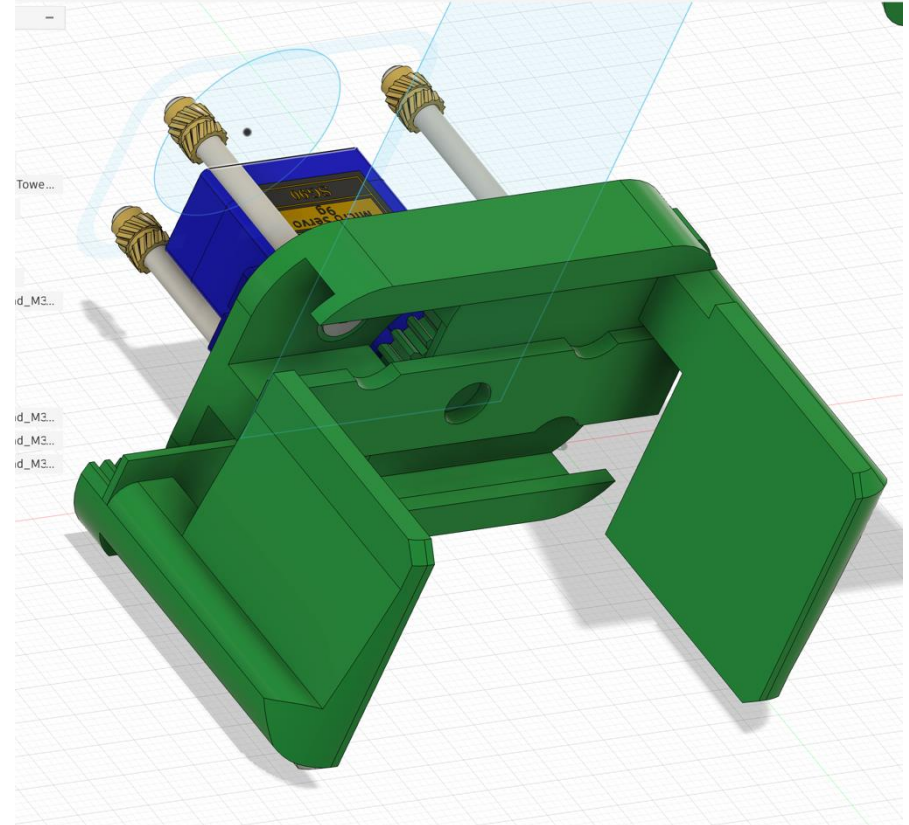
- The arm has gear teeth printed along its edge
- A pinion on the carriage turns and pushes the arm up or down
- 115 mm of travel, clears a standing king
- The gearbox holds the arm in place even when the motors are off



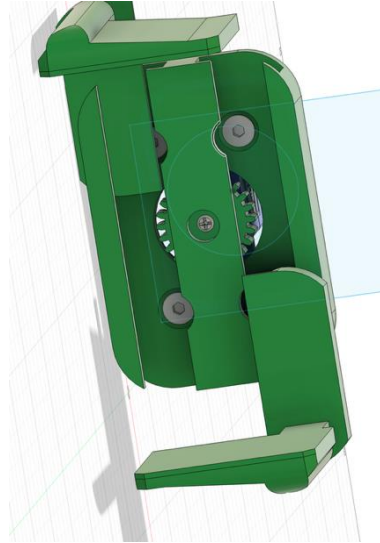
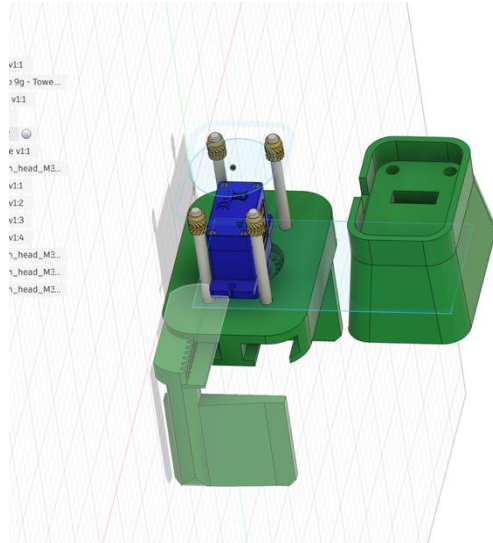
The gripper

Parallel jaws driven by one micro servo

- An SG90 servo turns a small pinion between two racks, one in each jaw
- The jaws slide straight towards each other, opening to 70 mm
- Thin fingers fit between neighbouring pieces on a full board
- Whatever it grips ends up centred, so the piece is placed centred



The gripper



Software

System architecture

Voice > Raspberry Pi 5 > ESP32 (FluidNC) > motors and claw

1. Everything on the Pi runs as one Python process.
2. The ESP32 receives G-code over USB serial. It does not decide anything.
3. Each stage can be developed and tested with the next one absent.

One turn, end to end

Player side

1. You say "pawn to e4"
2. Offline speech to text transcribes
3. Phonetic match against legal moves
4. Read back: "You said pawn to e4"
5. Gantry moves your piece

Robot side

6. Stockfish computes a reply
7. Read back: "AI plays knight to f6"
8. Gantry moves the robot piece
9. Listening again

The orchestrator: one hub, four modules

The rule

The orchestrator is the only module that calls another module.
Voice never talks to chess. Motion never touches the serial port.

Why we choose it?

Every team member owns one module and one function signature.
The orchestrator imports them, parses their output and feeds it to the next one.
Nobody has to read anybody else's code, and nobody is blocked waiting.

The turn state machine

State	What happens	Owner
LISTEN	Hand the current legal moves to the voice module and wait for speech	voice_matching
PARSE	Classify what was heard: <i>legal</i> , <i>illegal</i> or <i>unrecognised</i>	voice_matching
CONFIRM	Validate, apply the move, read it back out loud	chess_ai
MOVE (human)	Plan the G-code, stream it, block until the gantry stops	motion + orchestrator
THINK	Stockfish computes the reply	chess_ai
MOVE (AI)	Same plan and stream path for the robot's piece	motion + orchestrator

The Raspberry Pi and the link to the ESP32

Pi OS Lite, 64 bit

No desktop, no sound server. The board has no screen, so nothing is spent on one. Bare ALSA means audio routing is explicit, not decided by a daemon. Lite was chosen for RAM and predictability, not just boot speed.

The serial link

Pi > USB serial 115200 > ESP32 (FluidNC) > four drivers > motors

Payload is plain G-code, one line at a time, with an confirmation per line.
ok means queued, not finished. Status is polled until *Idle* before the game listens again.
With no ESP32 attached the lines are printed instead of sent, so motion was developed before the hardware existed.

The whole loop runs offline. Only the 40 MB speech model ever needed internet.

One process, five modules

voice_matching

Vosk speech to text and phonetic matching

chess_ai

python-chess, Stockfish, text to speech

motion

square to millimetres to G-code

orchestrator

turn state machine and the ESP32 serial link

main.py

entry point, wires the modules together

Team and roles

Role 1 Voice matching (David G.)

Vosk STT, phonetic move matching

Role 2 Chess AI (Dorde)

Rules authority, Stockfish, speech

Role 3 Motion planning (Eni)

Square to XY to G-code, collision avoidance

Role 4 Hardware (David M.)

FluidNC firmware, wiring, motor configuration

Role 5 Orchestration (Anton)

Turn state machine, serial link

The voice problem

Spoken chess letters are highly confusable: b d e g p t c z

- Over a microphone, "b4", "d4", "e4" and "g4" are nearly identical.
- Background noise and accents make it worse.
- One wrong letter is a completely different move, or an illegal one.

Why free transcription fails

Open vocabulary

The recogniser can return any word in the language. Nothing constrains it to chess.

A single misheard consonant produces a valid but wrong move, or nonsense.

Constrained matching

At any position there are only 20 to 40 legal moves. We compare what was heard against that short list.

The closest sounding candidate wins.

The constraint that saves us

20 to 40

legal moves at any given position

The rules engine generates them every turn.
Each is rendered as phrases a person might say.
For example "e4", "pawn to e4", "pawn e4".

We never transcribe freely.

The board always knows what is possible.

The recogniser only has to be close, not correct.

Vosk: offline speech to text

- 40 MB model which runs entirely on the Raspberry Pi.
- No cloud service, so no network latency.
- No audio ever leaves the device.
- Falls back to typed input if the model or microphone is missing.

Spoken read back

PLAYER

"pawn to e4"

BOARD

"You said pawn to e4"

The safety net

Every move is confirmed out loud before the gantry moves anything.

It catches the rare mismatch before a piece is physically moved.

Rules engine: python-chess

- Holds the single source of truth for board state.
- Generates the legal move list the voice layer matches against.
- Validates every move before it reaches the motion planner.
- Handles castling, en passant, promotion and check detection.

An illegal move can never reach the motors!

The opponent: Stockfish

Skill 0 to 20, default 5 Think time 0.5 s Turn limit 40

- Installed as a system binary
- Skill level is a command line flag, so difficulty is adjustable mid demo.

Text to speech

espeak

Raspberry Pi and Linux, needs espeak-ng

pyttsx3

Windows and Linux, pip package

piper

Pi and Linux, natural sounding, needs a model file

print

Everywhere, the default, needs nothing

Motion Planning and G-Code

Motion planning: goal and workflow

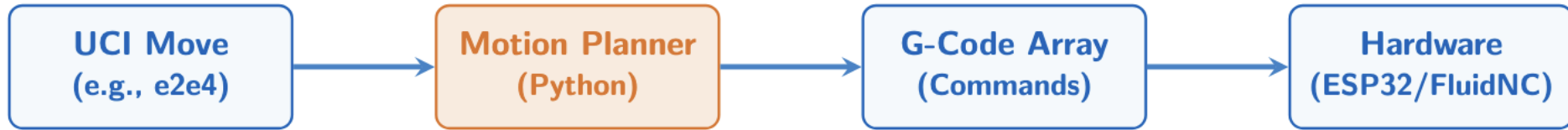
Goal

Translate abstract chess logic into precise physical machine commands.

Tech stack

Pure Python, object oriented design, GRBL and FluidNC G-code dialect.

Workflow



Architecture: separation of concerns

`planner.py` (logic)

Contains the core algorithms and pick-and-place sequence generation.

Hardware agnostic.

Never needs editing after deployment.

`config.py` (calibration)

Contains all physical constants.

Board dimensions, axis limits and piece heights.

The hardware team tunes the machine by editing variables, not logic.

The coordinate system and math

Mapping

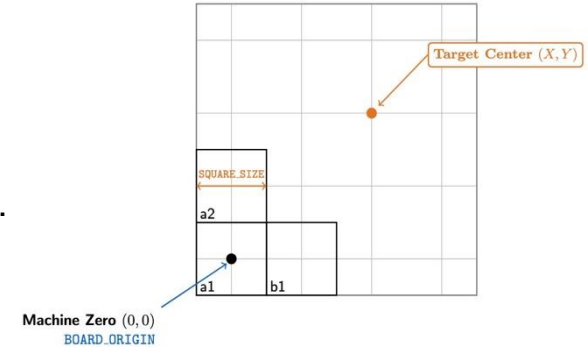
Translating an 8 by 8 matrix onto a physical Cartesian plane in millimetres.

Targeting

The machine origin (X0, Y0) is calibrated directly at the centre of square a1.

$$X = \text{BOARD_ORIGIN_X} + \text{Column_Index} \times \text{SQUARE_SIZE}$$

$$Y = \text{BOARD_ORIGIN_Y} + \text{Row_Index} \times \text{SQUARE_SIZE}$$



Safety first: moves outside the 535 x 545 mm envelope are rejected before any code is generated.

Collision avoidance: dynamic Z heights

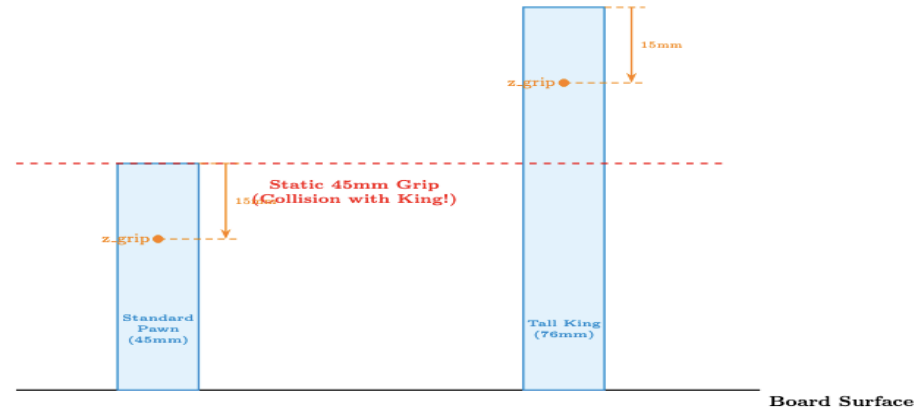
The problem

A static grip height of 45 mm will crush taller pieces such as the 76 mm King.

The solution

The planner maps each piece to a specifically measured Z-depth and A-axis servo angle using predefined profiles (e.g., Pawn: Z 85.0, A 62.0).

The Z-axis is positive-down, with Z0 at the safe top.



Adaptive carrying

Low carry

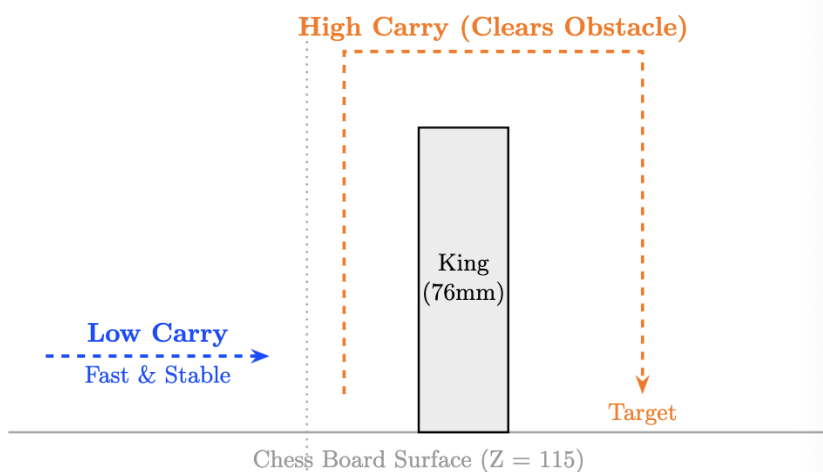
Used for sliding pieces travelling along empty paths. Keeps the piece close to the board, which is faster and more stable.

High carry

Above 76mm, used for knights jumping over occupied squares.

Clears the tallest piece on the board.

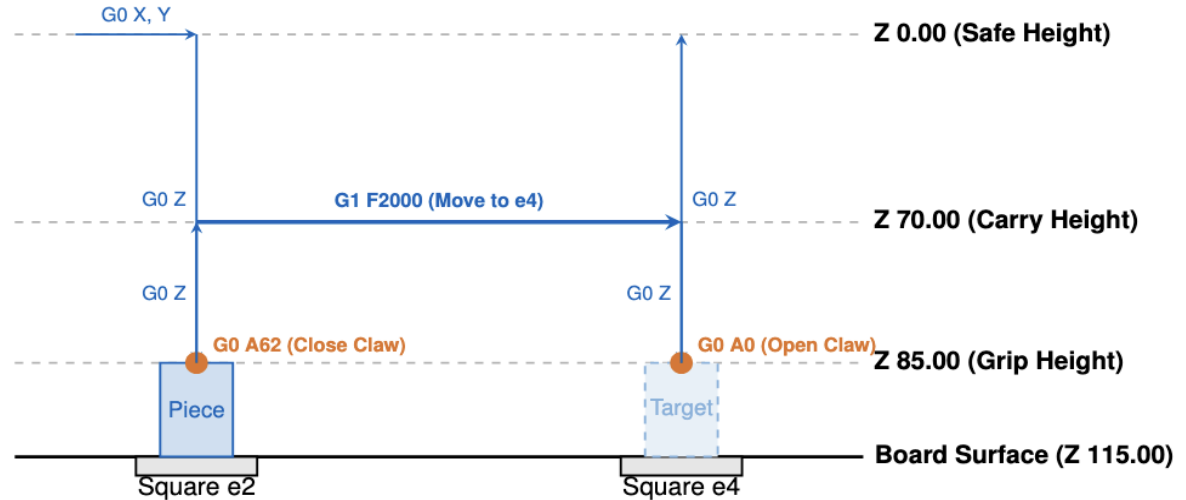
The planner chooses the carry height from the path, not from the piece alone.



Pick and place G-code

Translating the math into safe, executable GRBL and FluidNC instructions.

```
G0 Z0.00      ; raise to safe height(Z0 is TOP)
G0 X102.00 Y102.38 ; move above e2
G0 Z85.00     ; descend to grip pawn
G0 A62.00     ; close claw for pawn
G4 P0.5       ; let the servo settle
G0 Z70.00     ; low carry (15mm lift)
G1 X102.00 Y217.13 F2000 ; carry to e4
G0 Z85.00     ; descend to place
G0 A0.00      ; open claw
G4 P0.5       ; let the servo settle
G0 Z0.00      ; retract to safe height
```



Handling chess edge cases

Beyond standard moves, the planner intercepts and safely routes complex chess logic.

Captures

Generates a pre-sequence that lifts the opponent piece into an off-board graveyard matrix before moving the player piece.

Castling

A multi-part operation. The King slides on a low carry path, then the Rook takes a high lift path to cross over the newly placed King.

Pawn promotion

Routes the pawn to the graveyard and fetches a Queen of the correct colour from an off-board reserve row.

Hardware integration and calibration

Three manual measurements

BOARD_ORIGIN_X & Y: Set to 0.0, because machine zero (G54) is initialized exactly directly above the centre of chess square a1.

Z_BOARD: The absolute clearance (115mm) from the fully retracted Z0 top position down to the board surface.

Result

Once these three values are measured and set, the entire 535x545 mm Cartesian envelope is calibrated without changing a single line of logic.

Firmware and Integration

From G-code to motion

- Voice → Pi 5 → ESP32 + FluidNC → motors and claw
- Input: G-code lines over USB serial from the orchestrator
- Output: physical motion, plus the config and tooling that make it trustworthy

The ESP32 runs FluidNC

- ❑ FluidNC is proven open source CNC firmware for the ESP32.
- ❑ It handles step generation, acceleration and homing.
- ❑ We send motion intent as G-code, not individual step pulses.

- ❑ Firmware flashed once (FluidNC v4.0.3); config.yaml uploaded separately to the ESP32's filesystem

Why not custom firmware

Writing our own step timing would have consumed the whole project and produced something worse. Using FluidNC let us spend that time on the voice problem instead

Pi to ESP32 link

Transport

USB serial

Payload

one G-code line per message, e.g. G0 X100 Y85

Handshake

1. each line gets a tolerant ok (queued, not executed)
2. then ? until <Idle>
3. next line

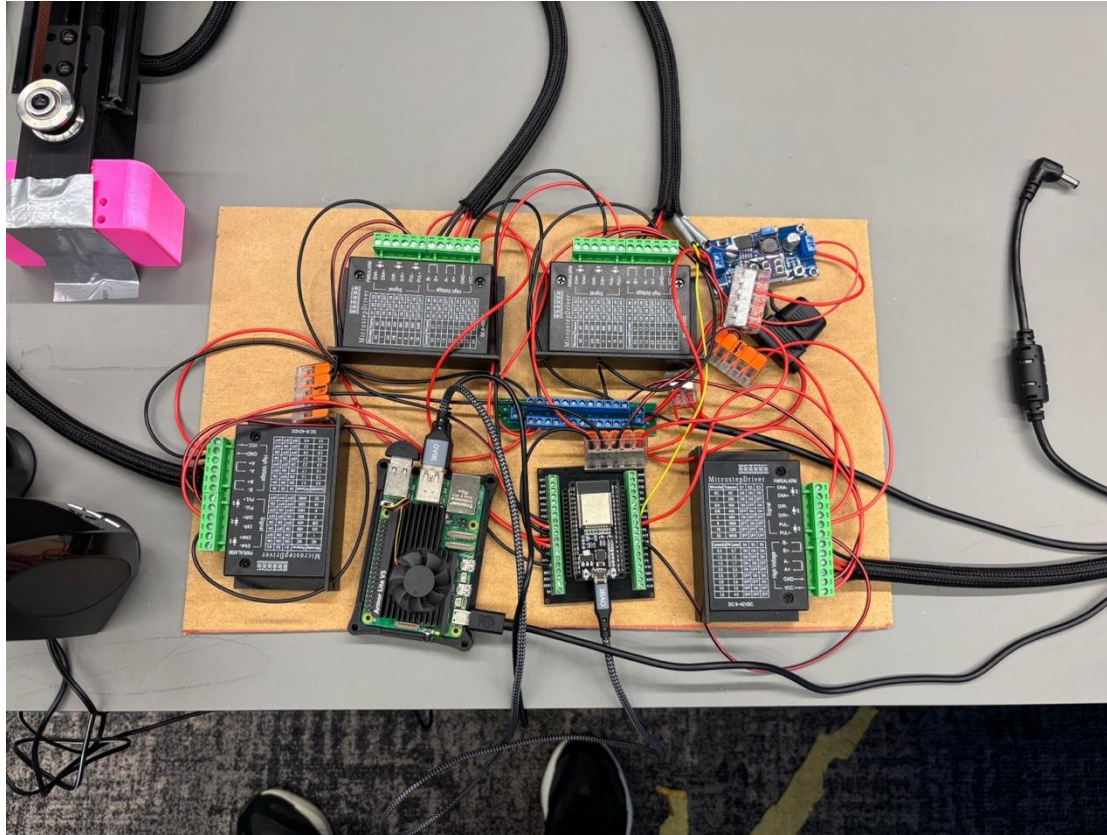
Error Handling

Testing without hardware

- pytest suites covering voice matching, chess rules and motion.
- Text mode replaces the microphone with typed input
- Scripted mode replays a fixed move list with no human at all
- Serial stub mode prints G-code instead of sending it

Electronics

Electronics overview



Power architecture

- A 24V brick supply
- One buck converter deriving the 4.8V for the servo inside the gripper
- The Raspberry Pi keeps its own supply, isolated from the motor power; it powers the ESP32 via microUSB
- All grounds meet at a single star point at the distributor and ESP32 GND pin (joined using WAGO connectors instead of soldering)

Sizing the supply

Power supply has a limit of 10A, more than enough for running all motors at the same time:

- 3 Nema 23 @ 2.5A
- 1 Nema 17 @ 0.5A
- SG90 servo using max 0.75A

Max Amps usage: 8.75A; so completely safe!

(we also have a fuse inlined just as an extra precaution ☺)

The 24 V bus

Supply > fuse > distribution block > drivers and bucks

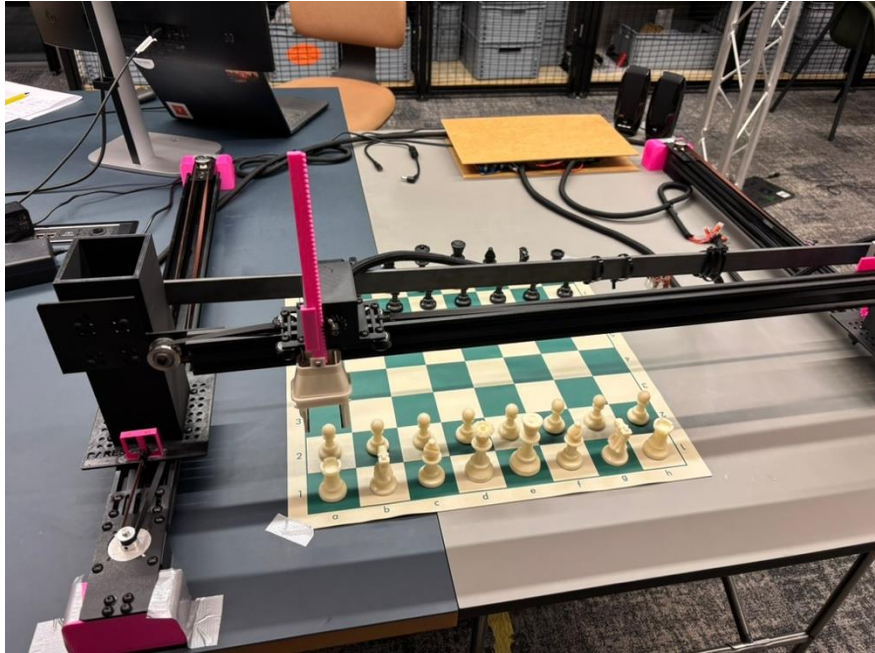
- ❖ A two bar distribution block turns one supply pair into many branches.
- ❖ Every driver and buck taps the same two bars, + and -

Conclusion and some recordings

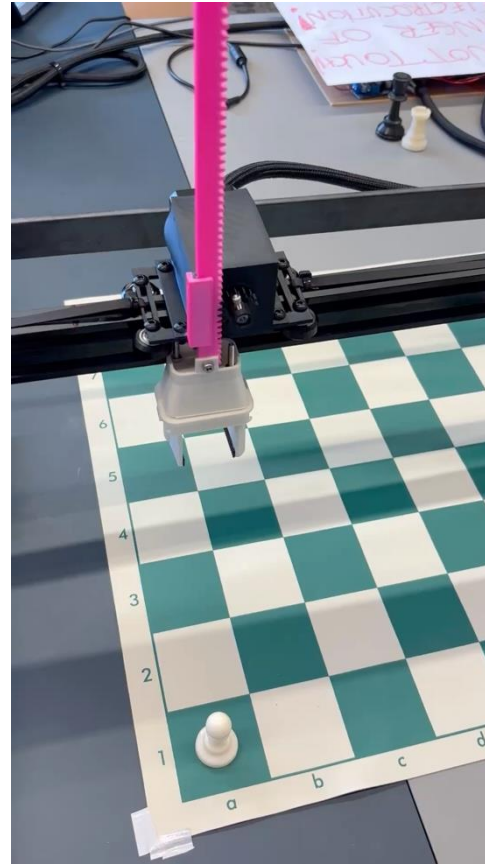
What it took to make it possible

- 683 euros funding spent on electronics, cables, equipment, V-rail slots, screws, GT2 belts etc.
- many hours of work, breaking components, reprinting and retrying :)
- we were lucky enough to be able to print most of the components and avoiding spending money doing so (special thanks to NXTSpace!)

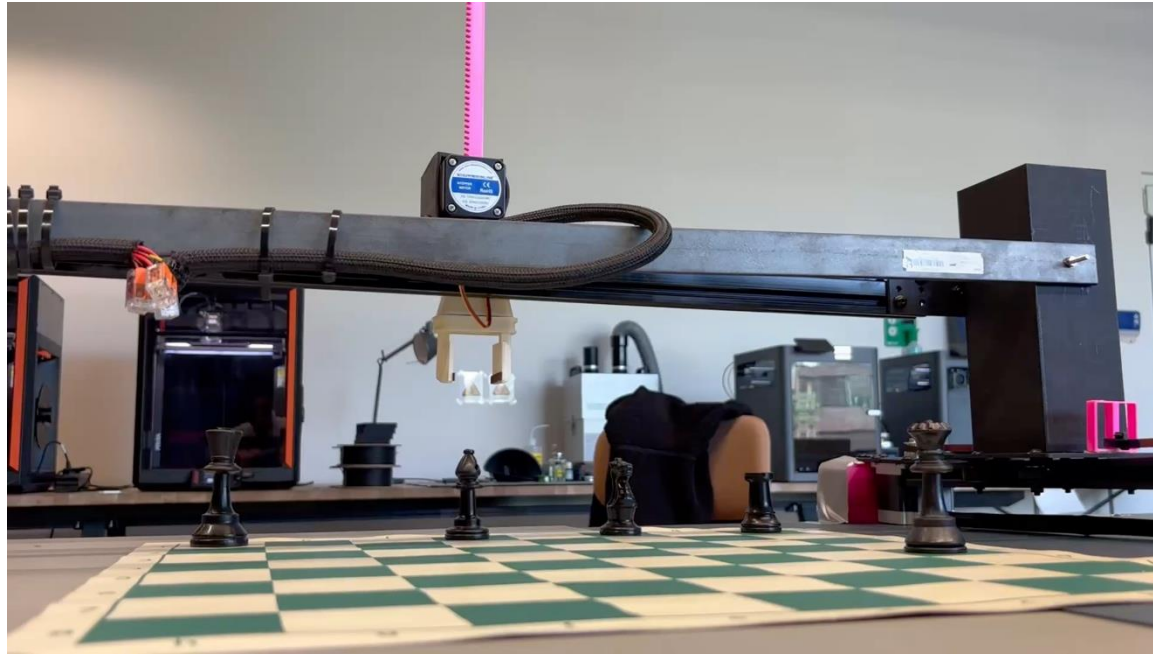
The finished prototype



Calibration



Testing movement of pieces



Beginning of the chess game



Moving the bishop and the knight



Kingside castle



Thanks for listening!
Questions?